Documentación de la API de ExpoTec

Backend Node.js y Express

Proyecto ExpoTec

E.E.S.T. N° 5 Amancio Williams

Versión de referencia 0.5.1

29 de octubre de 2026

Índice

1. Descripción general 3
2. Estructura del backend 3
3. Convenciones de la API 3
3.1 Formato de error 4
4. Autenticación y sesiones 4
4.1 Accesos sociales 5
5. Ejemplo de consumo 6
6. Roles y permisos 7
7. Endpoints 7
7.1 Salud 7
7.2 Autenticación (/api/auth) 8
7.3 Recuperación de contraseña (/api/auth/recovery) 9
7.4 Usuarios y permisos (/api/admin) 10
7.5 Espacios (/api/spaces) 10
7.6 Proyectos (/api/projects) 12
7.7 Inventario (/api/inventory) 14
7.8 Préstamos (/api/loans) 15
7.9 Galería (/api/gallery) 16
7.10 Fotos (/api/uploads) 17
8. Códigos de error 18
9. Base de datos 19
10. Pruebas 19
11. Compatibilidad con la APK 0.4.0 20

1. Descripción general

La API de ExpoTec es el backend que usan la aplicación Android y la versión web. Está hecha con Node.js 24 y Express 5, guarda los datos en una base MySQL (TiDB Cloud en producción) y protege las operaciones con JWT de acceso y de renovación. Este documento describe cómo interactuar con ella: convenciones, autenticación, roles, endpoints, respuestas y errores. La instalación del entorno está en la Guía de instalación y configuración.

- Dirección en producción: `https://expotec-api-gn19.vercel.app`.
- Dirección en desarrollo: `http://localhost:3001` (variable `PORT`).
- Versión del contrato: 0.5.1. Las consultas usan POST con los datos en el cuerpo JSON; para crear se usa POST, para reemplazar PUT, para cambios parciales PATCH y para bajas DELETE.

2. Estructura del backend

Cada solicitud sigue el mismo recorrido: ruta, validación de los datos recibidos, repositorio con la transacción SQL y respuesta JSON. Los permisos generales se comprueban en el middleware; las reglas finas, como ser integrante del proyecto o profesor asignado, se comprueban dentro de la misma transacción que modifica los datos.

```
backend/
  src/
    app.mjs            Armado para producción (Vercel)
    server.mjs         Servidor local (npm run dev)
    create-app.mjs     Express: seguridad, CORS, rutas y errores
    config/            Variables de entorno y conexión MySQL
    middleware/        Sesión, rol administrador, límite de intentos
    routes/            Rutas por dominio
    services/          JWT, contraseñas, accesos sociales, fotos
    repositories/      Consultas SQL y transacciones
    validators/        Validación de cada operación
  sql/                 Migraciones 001 a 010
  scripts/             Migrar, comprobar base, crear administrador
  test/                Pruebas unitarias y de integración
```

3. Convenciones de la API

- Formato: JSON (`Content-Type: application/json`) en solicitudes y respuestas.
- Métodos: `POST` consulta (rutas terminadas en `/query`) o crea, `PUT` reemplaza, `PATCH` modifica parcialmente y `DELETE` da de baja.
- Sin GET: la API no usa GET para leer datos. Las consultas se hacen con `POST` y sus filtros (`limit`, `offset`, `day`...) viajan en el cuerpo JSON, nunca en la URL; así no quedan registrados en historiales ni registros del servidor. Un campo desconocido responde 400.

- Excepciones: solo dos rutas responden a GET porque no las llama la app sino el navegador o el teléfono: la imagen publicada (`/api/uploads/:id/image`, que muestran las etiquetas de imagen) y el retorno OAuth (`/api/auth/social/:provider/callback`, al que redirigen Google, Discord y GitHub según el estándar OAuth 2.0). Ninguna de las dos recibe datos privados.
- Paginación: `limit` (1–100, por defecto 25) y `offset` (0–100000, por defecto 0).
- Identificadores: UUID v4. Un id con otro formato responde 400.
- Control de cambios simultáneos: los recursos editables tienen `version`. `PUT`, `PATCH` y `DELETE` deben enviar en el cuerpo la versión leída; si otra persona modificó el registro, responde 409 y el cliente conserva su borrador.
- Bajas lógicas: `DELETE` no borra el historial: marca el registro como dado de baja o cancelado.
- Validación: todos los campos se validan en el servidor aunque el formulario ya lo haya hecho. Un campo no admitido responde 400.
- Tamaño máximo del cuerpo: 16 KB en general, 64 KB en proyectos y 128 KB en espacios (413 si se supera).
- Límite de intentos: respuesta 429 con encabezado `Retry-After` (segundos).
- Caché: todas las respuestas usan `Cache-Control: no-store` e incluyen `X-Request-Id`.

3.1 Formato de error

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Revisá los campos indicados.",
    "fields": { "password": "Usá entre 12 y 128 caracteres." }
  },
  "requestId": "4d0f2c6e-0b1e-4d2a-9b7f-6c1f0c2a9e51"
}
```

`fields` aparece solo en errores de validación de formularios. `message` está en español y se puede mostrar directamente.

4. Autenticación y sesiones

Se usan dos tokens JWT firmados con HS256:

Tabla 1

Tokens de sesión, duración y uso

Token	Duración	Dónde vive	Uso
Acceso (`accessToken`)	15 minutos	Memoria de la app	Encabezado `Authorization: Bearer <token>` en rutas protegidas.
Renovación (`refreshToken`)	Sesión de hasta 30 días	Android: SecureStore. Web: cookie HttpOnly	Pedir un acceso nuevo con `POST /api/auth/refresh`.

- Cada renovación rota el token: el anterior deja de servir. Reutilizar uno ya usado revoca toda la sesión.
- La base guarda solo el hash SHA-256 del token de renovación, nunca el token.
- Las contraseñas se guardan con scrypt (N=131072, r=8, p=1) y sal aleatoria.
- El ingreso fallido responde siempre "El correo o la contraseña no son correctos", exista o no la cuenta.
- La app restaura la sesión al abrirse con el token de renovación guardado (sesión persistente).

Respuesta de registro, ingreso o renovación:

```
{
  "user": {
    "id": "UUID",
    "email": "alumno@escuela.edu.ar",
    "firstName": "Lucía",
    "lastName": "Fernández",
    "roles": ["student"],
    "createdAt": "2026-10-20T13:00:00.000Z"
  },
  "accessToken": "JWT",
  "refreshToken": "JWT de renovación",
  "tokenType": "Bearer",
  "expiresIn": 900
}
```

En la variante web (`/api/auth/browser/...`) la respuesta no incluye `refreshToken`: el servidor lo envía en una cookie HttpOnly, `SameSite=Strict`, que JavaScript no puede leer. Esas rutas exigen un encabezado `Origin` autorizado (protección CSRF).

4.1 Accesos sociales

ExpoTec admite Google, Discord y GitHub con OAuth 2.0 (PKCE en Google y GitHub). En Android, Google usa el selector nativo de cuentas; Discord y GitHub abren el navegador y vuelven a la app con el enlace `expotec://auth`.

Flujo: el cliente genera una prueba secreta y envía su hash al iniciar; el proveedor vuelve a `/api/auth/social/:provider/callback`; el cliente consulta el estado con la prueba y completa el acceso. Según el caso:

- `login`: la identidad ya está vinculada → se abre la sesión.
- `link`: el correo ya tiene cuenta → se pide la contraseña de ExpoTec para vincular.
- `signup`: cuenta nueva → se completa nombre, apellido, tipo de cuenta y contraseña propia.

Solo se aceptan correos verificados por el proveedor. No se guardan tokens de los proveedores.

5. Ejemplo de consumo

El siguiente ejemplo en PowerShell inicia sesión, guarda el token de acceso y consulta los proyectos visibles para la cuenta. Las mismas llamadas pueden hacerse desde cualquier cliente HTTP que envíe JSON.

```
$api = 'https://expotec-api-gn19.vercel.app'
$login = Invoke-RestMethod -Method Post -Uri "$api/api/auth/login" `
  -ContentType 'application/json' `
  -Body (@{ email = 'alumno@escuela.edu.ar'; password = 'Contraseña de ejemplo' } |
ConvertTo-Json)
$headers = @{ Authorization = "Bearer $($login.accessToken)" }
Invoke-RestMethod -Method Post -Uri "$api/api/projects/query" `
  -Headers $headers -ContentType 'application/json' `
  -Body (@{ limit = 25; offset = 0 } | ConvertTo-Json)
```

Cuando el token de acceso vence (15 minutos), el cliente pide uno nuevo con `POST /api/auth/refresh` enviando el token de renovación. La aplicación lo hace sola y repite la operación que había fallado.

6. Roles y permisos

Tabla 2

Roles de usuario y permisos principales

Rol	Código	Permisos principales
Visitante sin cuenta	—	Ver espacios, proyectos confirmados, galería aprobada y presentación TV.
Invitado	`guest`	Enviar fotos a la galería y ver el estado de sus aportes.
Alumno	`student`	Presentar, editar y cancelar sus propuestas; buscar compañeros y profesores; pedir elementos.
Profesor	`teacher`	Revisar las propuestas que lo tienen asignado: confirmar o pedir cambios.
Encargado de préstamos	`loans`	Ver inventario; registrar entregas, devoluciones y novedades.
Administrador	`admin`	Gestionar usuarios y roles, espacios y planos, inventario, moderación y reapertura de proyectos.

Una cuenta puede tener varios roles. El registro solo permite elegir `student` o `guest`; los demás roles los asigna un administrador. La API impide quitar el último administrador activo.

7. Endpoints

En las tablas de esta sección, la columna «Acceso» indica quién puede usar la operación: *Público* (sin token), *Sesión* (cualquier cuenta con token válido) o el rol requerido.

7.1 Salud

Tabla 3

Endpoints de estado del servicio

Método	Ruta	Acceso	Respuesta
POST	`/api/health`	Público	200 `{ status: "ok", service: "expotec-api" }` (no consulta la base).
POST	`/api/health/ready`	Público	200 `{ status: "ok", database: "connected" }`; 503 si la base no responde.

7.2 Autenticación (`/api/auth`)

Tabla 4

Endpoints de autenticación y accesos sociales

Método	Ruta	Acceso	Cuerpo / parámetros	Respuesta
POST	`/api/auth/register`	Público	`firstName`, `lastName`, `email`, `password` (12–128), `accountType` opcional (`student` o `guest`)	201 sesión; 409 `EMAIL_REGISTERED`
POST	`/api/auth/login`	Público	`email`, `password`	200 sesión; 401 `INVALID_CREDENTIALS`
POST	`/api/auth/refresh`	Token de renovación	`refreshToken`	200 sesión nueva; 401 `SESSION_INVALID`
POST	`/api/auth/logout`	Token de renovación	`refreshToken`	204
POST	`/api/auth/session`	Sesión	—	200 `{ user }` con roles actuales
POST	`/api/auth/browser/register`	Público (web)	igual que register	201 sesión + cookie
POST	`/api/auth/browser/login`	Público (web)	`email`, `password`	200 sesión + cookie
POST	`/api/auth/browser/refresh`	Cookie	`{}`	200 sesión + cookie rotada
POST	`/api/auth/browser/logout`	Cookie	`{}`	204, borra la cookie
POST	`/api/auth/social/providers/query`	Público	—	200 `{ providers: [{ id, name }] }` configurados
POST	`/api/auth/browser/social/start`	Público (web)	`provider`, `challenge` (SHA-256 hex)	200 `{ id, url }` del proveedor
GET	`/api/auth/social/:provider/callback`	Proveedor OAuth (excepción, ver Convenciones)	`state`, `code`	303 a la web con `#social=<id>`

Método	Ruta	Acceso	Cuerpo / parámetros	Respuesta
POST	`/api/auth/social/status`	Público	`id`, `proof`	200 `{ provider, email, mode }`
POST	`/api/auth/social/complete`	Público	`id`, `proof`, `mode` y según el modo `password`, `firstName`, `lastName`, `accountType`	200 sesión
POST	`/api/auth/social/google/native/start`	Público (Android)	`challenge`	200 `{ id, nonce, webClientId }`
POST	`/api/auth/social/google/native/verify`	Público (Android)	`id`, `proof`, `idToken`	200 `{ id }`

`social/status` y `social/complete` usan POST porque envían la prueba secreta del flujo: en una URL quedaría registrada en historiales y registros del servidor.

7.3 Recuperación de contraseña (`/api/auth/recovery`)

Tabla 5

Endpoints de recuperación de contraseña

Método	Ruta	Acceso	Cuerpo	Respuesta
POST	`/api/auth/recovery/query`	Público	—	200 `{ enabled }` (si el correo está configurado)
POST	`/api/auth/recovery/request`	Público	`email`	202 mensaje genérico (no revela si la cuenta existe)
POST	`/api/auth/recovery/complete`	Público	`token`, `password`, `confirmation`	200; cierra todas las sesiones de la cuenta

El enlace vence a los 20 minutos y sirve una sola vez. Límites: 3 pedidos por correo y 15 por IP cada hora.

7.4 Usuarios y permisos (`/api/admin`)

Tabla 6

Endpoints de administración de usuarios y roles

Método	Ruta	Acceso	Cuerpo / parámetros	Respuesta
POST	`/api/admin/users/query`	Administrador	`search` (nombre o correo, hasta 100), `limit`, `offset`	200 `{ users }`
POST	`/api/admin/users/:id/query`	Administrador	—	200 `{ user }`; 404 `USER_NOT_FOUND`
PUT	`/api/admin/users/:id/roles`	Administrador	`roles` (lista sin repetir de `student`, `teacher`, `loans`, `admin`, `guest`), `expectedRoles` (roles leídos antes de editar)	200 `{ user }`; 409 `ROLES_CHANGED` o `LAST_ADMIN`

7.5 Espacios (`/api/spaces`)

Tabla 7

Endpoints de espacios y planos

Método	Ruta	Acceso	Cuerpo / parámetros	Respuesta
POST	`/api/spaces/query`	Público	—	200 `{ spaces }` con `furnitureCount` y `boothCount`
POST	`/api/spaces/:id/query`	Público	—	200 `{ space }` con `furniture` y `booths` (puesto y proyecto confirmado)
POST	`/api/spaces`	Administrador	`name` (2–80), `description` (hasta 150)	201 `{ space }`; 409 `SPACE_LIMIT` (100 activos)
PUT	`/api/spaces/:id`	Administrador	`version`, `name`, `description`, `furniture`, `boothFurnitureIds`	200 `{ space }`; 409 `SPACE_CHANGED`, `SPACE_IN_USE`, `FURNITURE_ID_RESERVED`
DELETE	`/api/spaces/:id`	Administrador	`version`	204; 409 `SPACE_IN_USE` si tiene proyectos vigentes

Cada pieza de `furniture` es `{ id, type, x, y, rotation }` con `type` en `table`, `bench`, `power` o `label`; `x` e `y` en porcentaje (0–100) y `rotation` entero 0–359. Las mesas agregan `width` (6–30) y `height` (4–25); los rótulos, `text` (2–60). `boothFurnitureIds` indica qué mesas son puestos inscribibles. Máximo 150 piezas.

7.6 Proyectos (`/api/projects`)

Tabla 8

Endpoints de proyectos y revisiones

Método	Ruta	Acceso	Cuerpo / parámetros	Respuesta
POST	`/api/projects/public/query`	Público	`limit`, `offset`	200 `{ projects }` confirmados
POST	`/api/projects/public/:id/query`	Público	—	200 `{ project }` sin datos privados; 404 si no está confirmado
POST	`/api/projects/presentation/query`	Público	`limit` (1–100), `after` (cursor UUID)	200 `{ projects, nextCursor }` para la presentación TV
POST	`/api/projects/query`	Sesión	`limit`, `offset`	200 `{ projects }`: propios (alumno), asignados (profesor) o todos (admin)
POST	`/api/projects/people/find`	Alumno o admin	`email`, `role` (`student` o `teacher`)	200 `{ person }`; 404 `PERSON_NOT_FOUND`
POST	`/api/projects/:id/query`	Integrante, profesor asignado o admin	—	200 `{ project }` con revisiones y elementos pedidos
POST	`/api/projects`	Alumno	propuesta (ver abajo)	201 `{ project }` en estado `pending`
PUT	`/api/projects/:id`	Integrante (alumno)	propuesta + `version`	200 `{ project }`; 409 `PROJECT_CHANGED`, `PROJECT_LOCKED`
POST	`/api/projects/:id/reviews`	Profesor asignado (`approve`, `changes`) o admin (`reopen`)	`version`, `action`, `note` (10–2000, obligatoria salvo para aprobar)	200 `{ project }`; 409 `BOOTH_OCCUPIED`, `RESOURCE_STOCK`, `PROJECT_HAS_LOANS`
DELETE	`/api/projects/:id`	Integrante o admin	`version`	200 `{ project }` cancelado; libera puesto y reservas

Cuerpo de una propuesta:

```
{
  "name": "Estación meteorológica",
```

```
  "description": "Medimos temperatura, humedad y presión durante el día.",
  "course": "6° 1° · Informática",
  "imageUrl": "https://expotec-web-gn19.vercel.app/api/uploads/<id>/image",
  "videoUrl": "",
  "isSoftwareProject": true,
  "githubUrl": "https://github.com/equipo/estacion",
  "websiteUrl": "",
  "memberIds": ["UUID del alumno"],
  "teacherIds": ["UUID del profesor"],
  "subjects": ["Programación", "Electrónica"],
  "boothId": "UUID del puesto",
  "resources": [{ "resourceId": "UUID", "quantity": 2 }]
}
```

Reglas: nombre 2–120, descripción 20–5000, curso 2–100; enlaces HTTPS; 1–12 integrantes alumnos (incluye a quien presenta) y 1–5 profesores; 1–10 materias; hasta 50 elementos. Estados: `pending` → `approved` / `changes` → `pending` (al reenviar) y `cancelled`. Confirmar asigna el puesto y reserva los elementos en una sola transacción: dos confirmaciones simultáneas no pueden ocupar el mismo puesto.

7.7 Inventario (`/api/inventory`)

Tabla 9

Endpoints de inventario

Método	Ruta	Acceso	Cuerpo / parámetros	Respuesta
POST	`/api/inventory/catalog/query`	Alumno, profesor, préstamos o admin	—	200 `{ resources }` activos, para pedir en propuestas
POST	`/api/inventory/query`	Préstamos o admin	`limit`, `offset`, `archived` (`true`/`false`)	200 `{ resources }`
POST	`/api/inventory/:id/query`	Préstamos o admin	—	200 `{ resource }` con historial (`events`)
POST	`/api/inventory`	Administrador	`name` (2–100), `description` (hasta 250), `total` (0–100000), `unavailable`, `note`	201 `{ resource }`; 409 `RESOURCE_NAME_EXISTS`
PUT	`/api/inventory/:id`	Administrador	lo anterior + `version`	200 `{ resource }`; 409 `RESOURCE_CHANGED`, `RESOURCE_IN_USE`
DELETE	`/api/inventory/:id`	Administrador	`version`	204; 409 `RESOURCE_RESERVED`, `RESOURCE_HAS_LOANS`

`resource` incluye `total`, `reserved`, `available`, `unavailable` (fuera de servicio), `outstanding` (sin devolver), `availableToLend`, `version` y `archivedAt`.

7.8 Préstamos (`/api/loans`)

Tabla 10

Endpoints de préstamos y devoluciones

Método	Ruta	Acceso	Cuerpo / parámetros	Respuesta
POST	`/api/loans/query`	Préstamos o admin	`limit`, `offset`, `day` (1–3), `openOnly`	200 `{ loans, summary: { pendingCount, pendingQuantity } }`
POST	`/api/loans/projects/query`	Préstamos o admin	`day` (obligatorio), `limit`, `offset`	200 `{ projects }` confirmados con lo que pueden retirar
POST	`/api/loans/:id/query`	Préstamos o admin	—	200 `{ loan }` con `returns`
POST	`/api/loans`	Préstamos o admin	`projectId`, `projectVersion`, `resourceId`, `day`, `quantity`, `recipientId`, `note`	201 `{ loan }`; 409 `ALREADY_DELIVERED`, `LOAN_STOCK`, `PROJECT_LOAN_LIMIT`
POST	`/api/loans/:id/returns`	Préstamos o admin	`version`, `quantity`, `damagedQuantity`, `note`	200 `{ loan }`; 409 `LOAN_CHANGED`, `RETURN_EXCEEDS_LOAN`

Solo hay una entrega por proyecto, elemento y jornada. Las devoluciones pueden ser parciales; una novedad sin unidades (`quantity: 0`) o unidades dañadas exigen una nota de 10 caracteres o más. Las unidades dañadas quedan fuera de servicio en el inventario.

7.9 Galería (`/api/gallery`)

Tabla 11

Endpoints de la galería de fotos

Método	Ruta	Acceso	Cuerpo / parámetros	Respuesta
POST	`/api/gallery/public/query`	Público	`limit`, `offset`	200 `{ photos }` aprobadas (sin correos)
POST	`/api/gallery/mine/query`	Sesión	`limit`, `offset`	200 `{ photos }` propias con estado y motivo
POST	`/api/gallery`	Sesión	`imageUrl` (HTTPS o imagen subida), `caption` (5–400)	201 `{ photo }` pendiente; 409 `GALLERY_PENDING_LIMIT` (20)
POST	`/api/gallery/moderation/query`	Administrador	`status` (`pending`, `approved`, `rejected`, `all`), `limit`, `offset`	200 `{ photos }`
PATCH	`/api/gallery/:id`	Administrador	`version`, `decision` (`approved` o `rejected`), `note` (obligatoria al rechazar)	200 `{ photo }`; 409 `PHOTO_CHANGED`

7.10 Fotos (`/api/uploads`)

Tabla 12

Endpoints de carga de fotos

Método	Ruta	Acceso	Cuerpo / parámetros	Respuesta
POST	`/api/uploads`	Sesión (alumno para `project`)	`purpose` (`gallery` o `project`), `contentType` (JPG, PNG o WebP), `size` (hasta 10 MB)	201 `{ id, pathname, clientToken }` válido 10 minutos
POST	`/api/uploads/:id/complete`	Dueño de la carga	—	200 `{ imageUrl }`; 422 `IMAGE_INVALID`
POST	`/api/uploads/preview`	Dueño o admin	`imageUrl`	200 `{ image }` en base64
GET	`/api/uploads/:id/image`	Público (excepción, ver Convenciones)	—	200 `image/jpeg` si la imagen está publicada; 404 si es privada

El cliente sube el archivo directamente al almacenamiento privado con `clientToken`; luego `complete` valida el contenido real, lo optimiza a JPEG de hasta 1600 px y devuelve la URL definitiva.

8. Códigos de error

Tabla 13

Códigos de error por estado HTTP

HTTP	Códigos	Significado
400	`VALIDATION_ERROR`, `INVALID_JSON`, `PARTICIPANTS_INVALID`, `RECIPIENT_INVALID`, `RESET_INVALID`, `SOCIAL_FAILED`, `IMAGE_NOT_AVAILABLE`	Datos inválidos o incompletos.
401	`SESSION_INVALID`, `INVALID_CREDENTIALS`	Sin sesión, token vencido o credenciales incorrectas.
403	`FORBIDDEN`, `ORIGIN_FORBIDDEN`	El rol no alcanza o el origen web no está autorizado.
404	`NOT_FOUND`, `USER_NOT_FOUND`, `SPACE_NOT_FOUND`, `PROJECT_NOT_FOUND`, `PERSON_NOT_FOUND`, `RESOURCE_NOT_FOUND`, `LOAN_NOT_FOUND`, `PHOTO_NOT_FOUND`, `IMAGE_NOT_FOUND`, `SOCIAL_DISABLED`	Ruta o registro inexistente (o sin acceso).
409	`EMAIL_REGISTERED`, `ROLES_CHANGED`, `LAST_ADMIN`, `SPACE_CHANGED`, `SPACE_IN_USE`, `SPACE_LIMIT`, `FURNITURE_ID_RESERVED`, `PROJECT_CHANGED`, `PROJECT_LOCKED`, `PROJECT_HAS_LOANS`, `PROJECT_NOT_APPROVED`, `PROJECT_LOAN_LIMIT`, `BOOTH_OCCUPIED`, `BOOTH_UNAVAILABLE`, `RESOURCE_CHANGED`, `RESOURCE_ARCHIVED`, `RESOURCE_NAME_EXISTS`, `RESOURCE_LIMIT`, `RESOURCE_RESERVED`, `RESOURCE_IN_USE`, `RESOURCE_HAS_LOANS`, `RESOURCE_STOCK`, `RESOURCE_UNAVAILABLE`, `RESOURCE_NOT_RESERVED`, `RESOURCE_STATE_CHANGED`, `ALREADY_DELIVERED`, `LOAN_CHANGED`, `LOAN_STOCK`, `RETURN_EXCEEDS_LOAN`, `PHOTO_CHANGED`, `PHOTO_ALREADY_REVIEWED`, `GALLERY_PENDING_LIMIT`, `SOCIAL_CHANGED`, `SOCIAL_CONFLICT`, `UPLOAD_CHANGED`	Conflicto con el estado actual (cambio simultáneo, stock, puesto ocupado, límites).
413	`BODY_TOO_LARGE`	El cuerpo supera el límite de la ruta.
422	`IMAGE_INVALID`	El archivo no es una imagen válida.
429	`RATE_LIMITED`	Demasiados intentos; esperar los segundos de `Retry-After`.
500	`INTERNAL_ERROR`	Error inesperado (se registra solo el `requestId`).
503	`DATABASE_UNAVAILABLE`, `RECOVERY_UNAVAILABLE`, `UPLOADS_UNAVAILABLE`, `UPLOAD_FAILED`	Servicio externo no disponible o sin configurar.

9. Base de datos

MySQL relacional y normalizada (InnoDB, `utf8mb4`). Las migraciones de `sql/` se aplican en orden con `npm run db:migrate`.

Tabla 14

Migraciones SQL y tablas que crean

Migración	Tablas
`001_auth.sql`	`users`, `roles`, `user_roles`, `sessions`, `refresh_tokens`, `auth_rate_limits`
`002_spaces.sql`	`spaces`, `furniture`, `booths`
`003_projects.sql`	`projects`, `project_members`, `project_teachers`, `subjects`, `project_subjects`, `booth_allocations`, `project_reviews`
`004_inventory.sql`	`resources`, `project_resource_requests`, `resource_allocations`, `inventory_events`
`005_loans.sql`	`loans`, `loan_returns`, `resource_conditions`
`006_gallery.sql`	`gallery_photos`, `gallery_reviews`
`007_social.sql`, `008_social_browser.sql`	`social_providers`, `social_identities`, `social_flows`
`009_uploads.sql`	`image_uploads`
`010_password_recovery.sql`	`password_resets`

Relaciones principales: un usuario tiene varios roles (`user_roles`); un proyecto tiene varios integrantes, profesores y materias (tablas de relación); un puesto (`booths`) es una mesa de un espacio y se asigna a un solo proyecto (`booth_allocations`); los préstamos registran proyecto, elemento, jornada y sus devoluciones. El diagrama entidad-relación está en la carpeta `docs/diagramas` del repositorio.

10. Pruebas

```
npm test                    # 60 pruebas unitarias, sin base de datos
npm run test:projects       # integración: requiere TEST_DB_NAME=expotec_test_...
```

Las integraciones (`test:database`, `test:spaces`, `test:users`, `test:gallery`, `test:projects`, `test:inventory`, `test:loans`, `test:presentation`, `test:recovery`, `test:social`, `test:uploads`) usan una base exclusiva de pruebas y nunca la de producción. `test:database` exige además una base vacía.

11. Compatibilidad con la APK 0.4.0

La APK 0.4.0 usa las rutas anteriores, todas con POST. Las consultas de entonces son exactamente las rutas actuales (`POST /api/spaces/query`, `POST /api/projects/people/find`, etc.), así que siguen funcionando sin cambios. Las escrituras cambiaron de método; `src/middleware/legacy-routes.mjs` traduce cada una a su equivalente actual antes de llegar a los enrutadores, sin duplicar lógica. Esas respuestas incluyen el encabezado `Deprecation: true`.

Tabla 15

Equivalencias entre rutas de escritura anteriores y actuales

Escritura anterior (POST)	Ruta actual
`/api/spaces/create`, `/:id/save`, `/:id/archive`	`POST /api/spaces`, `PUT /api/spaces/:id`, `DELETE /api/spaces/:id`
`/api/projects/create`, `/:id/save`, `/:id/review`, `/:id/cancel`	`POST /api/projects`, `PUT /:id`, `POST /:id/reviews`, `DELETE /:id`
`/api/inventory/create`, `/:id/save`, `/:id/archive`	`POST /api/inventory`, `PUT /:id`, `DELETE /:id`
`/api/loans/deliver`, `/:id/return`	`POST /api/loans`, `POST /api/loans/:id/returns`
`/api/gallery/submit`, `/:id/review`	`POST /api/gallery`, `PATCH /api/gallery/:id`
`/api/uploads/prepare`, `/complete`	`POST /api/uploads`, `POST /api/uploads/:id/complete`
`/api/admin/users/:id/roles/save`	`PUT /api/admin/users/:id/roles`

La APK 0.5.0 usaba GET para las consultas y no es compatible con esta versión: hay que reemplazarla por la 0.5.1. Cuando todos los teléfonos tengan la APK 0.5.1 o posterior, se puede quitar `app.use(legacyRoutesMiddleware)` de `create-app.mjs`.